Week 4

Aim

The aim of this seminar was to look more into malicious software, how it operates and how it can be detected. Through completing the workshop, my aim was to gain a better understanding of malware detection systems and how we can utilise this knowledge in the future to build applications that can protect users better.

Glossary

Anti-virus – A security program that is designed to bloc, detect and remove malware.

Hashing – Mathematical process that converts data into a fixed length string.

Comparison system – A method that compares two values for example comparing hashes against stored hash.

Malware connection – The communication channel used by malware to connect to a server and spread to other systems, this can include features such as infected file propagation.

Detection – The process of identifying malicious activity.

Signature – A unique pattern associated with a specific piece of malware.

Payload – Part of the malware that performs harmful action.

Worm – A self-replicating type of malware that spreads automatically across a network without requiring user intervention.

File integrity - A security feature that ensures that a file has not been tampered with.

Discussion Points

Section 1: Discussion points

- Why do security systems rely on file hashes instead of timestamps or file sizes?

This is because a hash is a unique value which represents the content of the file, therefore even if one bit of the file is changed, the entire hash value is then altered. This alteration allows for the users to be alerted that their file has been changed and tampered with and therefore the integrity of the file is questioned. Therefore, using hashing ensures strong file integrity that cannot be easily faked.

- How might malware attempt to evade integrity checking?

Malware may attempt to update by modifying their own configuration, they also might have rootkit behaviour which allows their changes to be hidden. Alongside with this, injecting malicious code into memory instead of altering files on a disk which is also known as fileless malware is one of the many ways malwares can evade detection.

- What happens if a legitimate system update changes many hashes — how should you manage that?

You should use a trusted updated source and implement change control workflows to verify an update before accepting them.

Section 2: Discussion points

- How could this approach complement antivirus scanning?

Antivirus detects known malware signatures and integrity monitors unauthorized changes and therefore combining both can increase the chances of detection as integrity systems serves as a second layer of validation that anti-viruses may miss.

- What are the limitations if a rootkit hides file access at the kernel level?

Integrity tools may receive falsified results since the rootkits can intercept the file system. They can also ensure that the hashing checks appear clean despite being infected an only hardware-based scanning cam bypass kernel level manipulation.

- How could you extend this system to automatically restore modified files from backup?

When suspicious activity is found, it can quarantine the altered file or replace it with a good well-known copy. Alongside with this, using version control and cryptographic signing you can ensure that the backups are also not tampered with.

Section 3: Discussion points

- Why is signature-based detection ineffective against polymorphic or metamorphic viruses?

  Polymorphic malware encrypts or rewrites its own code on each infection whilst metamorphic malware modifies its structure and logic whilst maintaining the same behaviours and therefore no two samples are the same rendering a signature-based system ineffective.

- Could attackers intentionally include harmless “signature-looking” strings to trigger false alarms?

  Yes, this is known as signature poisoning as attackers may fake patterns to overwhelm teams with false positives which than erodes trust in the security systems

- How do heuristic and behavioural scanners improve on this approach?

They analyse actions not static code, therefore ensuring that unusual behaviour is flagged, allowing for detection of new malware variants.

Section 4: Discussion Points

- How does the infection curve change if you double the number of attempts per host?

  The infection than spreads exponentially faster since doubling the attempts also doubles the probability of successfully compromising a host.

- Why might a worm choose a local subnet propagation strategy?

Local networks have weaker internal security as they have predictable Ip ranges, lower latency and firewall permissiveness.

- Which containment strategy (rate halting, scan detection, thresholding) would be most effective here?

Rate limiting: Slows the worm down

Scan detection detects unusual behaviour

Thresholding: Blocks hosts after a fixed number of outbound attempts.

Section 5: Discussion Points

- Which layer of defence failed first in most real-world malware outbreaks?

Patch management and vulnerability control fail first since a lot of systems exploit outdated systems.

- How can AI-based systems be trained to detect abnormal patterns safely?

Utilise large datasets of benign versus malicious behaviour an apply anomaly detection. It can also train models offline to avoid poisoned inputs, and Ai can also explain why a particular model is banned.

- How should security teams balance detection sensitivity and false positives?

  They should maintain regular review sessions of their security systems; they must also set limitations on environment behaviour and tune rules to gradually reduce alert fatigue. They should also have multilayer correlation so that high alerts are only triggered when multiple alerts are aligned.

Errors and Solutions

During the completion of the workshop, I ran into the challenge of understanding the countermeasure design challenge. Initially, I believed that I had to mathematically simulate the decrease in worm propagation however all that was required for that was one simple function which did not seem right. Additionally, the course worked asked to work in a team, I did team up with someone however we never managed to collaborate on the coursework together and therefore decided to complete the task alone. Eventually, during week 8 feedback, I asked and was recommended to create a brute force defence system which was what I was initially speculating on doing.

Key Takeaways

Malware is more than just code – Malware also consist of a wide variety of behavioural patterns which can be hard to identify with simple static code, with AI growing every day, the ability to combat this will also grow. Working with analysing static code also made me realise how sophisticated malware can become.

Network based threads spread faster than expected – During my brute force simulation and worm propagation coursework I realised how quickly an outbreak can spread without simple implementation of rate limiting. It highlighted to me how important network layering really is.

Detect and Monitor – From this workshop, I also took away how vital it is to keep monitoring and detection services on your system. Knowing what is being hosted on your system is so important and often just by simple updates to your network you can avoid attacks.